МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
**“КАЗАНСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛСКИЙ
ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ им. А.Н.
ТУПОЛЕВА-КАИ”**

Институт **компьютерных технологий и защиты информации**
Кафедра **Систем Информационной безопасности**

Отчет по лабораторной работе №3
По теме
“ Декомпозиция вычислений: распараллеливание цикла и функциональная декомпозиция”
По дисциплине
“Аппаратные средства телекоммуникационных систем”

Выполнил студент
Группы 4404
Сайдаков Л.И.

Казань, 2026

Цель работы. Изучение двух подходов к декомпозиции вычислений в параллельных программах для систем с общей памятью: декомпозиции по данным (распараллеливание цикла) и функциональной декомпозиции. Подходы рассматриваются на примере вычисления геометрического центра множества точек трёхмерного пространства, координаты которых считываются из файла. В ходе работы выполнена задача 5 методических указаний «Параллельное программирование на OpenMP». Работа выполняется на языке C# (платформа .NET), поэтому конструкции OpenMP моделируются стандартными средствами языка: объекты System.Threading.Thread соответствуют команде потоков, вызов Thread.Join для всех потоков — неявному барьеру в конце параллельной области, объединение частичных сумм потоков после их завершения — операции reduction, отдельные потоки для независимых вычислений — секциям (sections) и функциональной декомпозиции.

Задание:

Напишите программу, которая читает из файла координаты точек в 3D пространстве (x, y, z) и вычисляет геометрический центр, который есть среднее по x, y и z. Напишите две версии программы: одну с распараллеливанием цикла, другую с функциональной декомпозицией.

Общие требования: число потоков и размер задачи передаются программе через командную строку; запрещается использовать библиотечные функции, целиком решающие задачу (в частности, Parallel.For и PLINQ); каждый запуск, о котором говорится в задаче (разные версии программы), подтверждается скриншотом с видимой командой запуска и полным выводом

Ход работы. Исходные точки хранятся в текстовом файле: одна точка в строке, три числа x, y, z через пробел (десятичный разделитель — точка). Пустые строки и строки, начинающиеся с символа «#», пропускаются. При чтении координаты раскладываются в три отдельных массива X, Y и Z. Для получения больших входных данных программа имеет режим gen, который создаёт файл со случайными точками из диапазона [−100; 100]. Число потоков,

имя файла и версия программы (loop, functional или all) передаются через командную строку.

Перед параллельными вариантами выполняется последовательное вычисление, результат которого используется как эталон. Корректность параллельных версий проверяется сравнением с эталоном с относительной погрешностью 10^{-9}: порядок сложения чисел с плавающей точкой в потоках отличается от последовательного, поэтому строгое равенство последних разрядов не гарантируется. Перед замером времени выполняется прогрев (первый запуск вычисления не учитывается), чтобы время JIT-компиляции не попало в результат; время измеряется классом Stopwatch.

Версия 1 (loop) — распараллеливание цикла (декомпозиция по данным). Диапазон индексов точек делится между потоками на непрерывные равные части, остаток распределяется между первыми потоками. Каждый поток суммирует координаты только своих точек в локальных переменных и по окончании записывает три частичные суммы в свой элемент общего массива. После Join главный поток складывает частичные суммы и делит их на число точек N (аналог reduction). Гонки данных отсутствуют: массивы координат только читаются, а каждый поток пишет в собственную ячейку массива частичных сумм, поэтому мьютексы не требуются.

Версия 2 (functional) — функциональная декомпозиция. Работа делится не по данным, а по функциям: первый поток вычисляет среднее по x, второй — по y, третий — по z. Потоки независимы, читают только свой массив и пишут только в свою переменную, поэтому синхронизация также не нужна. Особенность версии: число потоков всегда равно трём и не зависит от значения, переданного в командной строке, то есть максимально возможное ускорение ограничено тремя.

Команды запуска:

```
cd lab3
dotnet run -c Release -- gen points.txt 10000000
dotnet run -c Release -- 8 sample_points.txt
dotnet run -c Release -- 8 points.txt loop
dotnet run -c Release -- 8 points.txt functional
dotnet run -c Release -- 8 points.txt all
```

На рисунке 1 показан запуск обеих версий на небольшом тестовом файле sample_points.txt, содержащем вершины единичного куба. Центр куба известен заранее: (0.5, 0.5, 0.5); обе версии выдали этот результат, совпадающий с эталоном (OK: true).

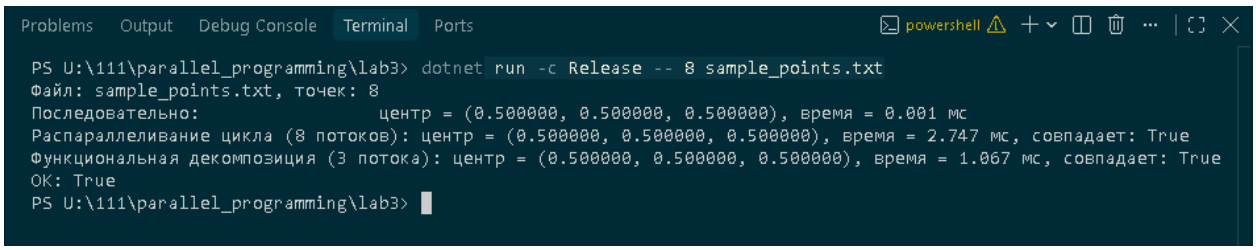


Рис. 1 – запуск на тестовом файле sample_points.txt (8 вершин единичного куба)

На рисунке 2 показано создание файла из 10 000 000 случайных точек командой gen и запуск версии с распараллеливанием цикла на 8 потоках. Видны команда запуска, вычисленный геометрический центр, время выполнения последовательной и параллельной версий и признак совпадения с эталоном.

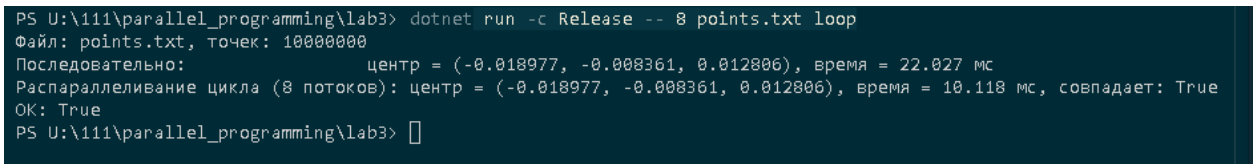


Рис. 2 – создание файла points.txt и запуск версии loop с 8 потоками

На рисунке 3 показан запуск версии с функциональной декомпозицией на том же файле. Версия использует три потока независимо от аргумента командной строки; результат совпадает с эталоном.

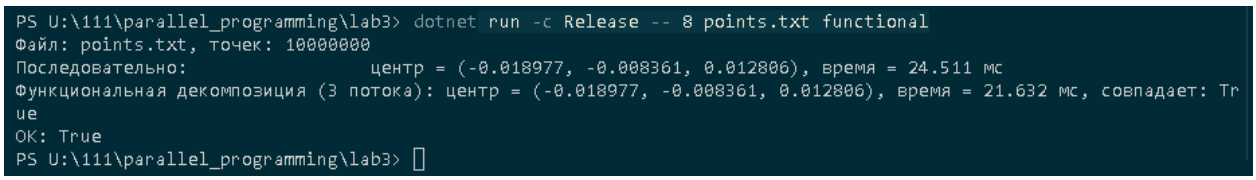


Рис. 3 – запуск версии functional

На рисунке 4 показан запуск обеих версий в одном процессе (режим all) для сравнения времени выполнения. Сводные результаты приведены в таблице 1.

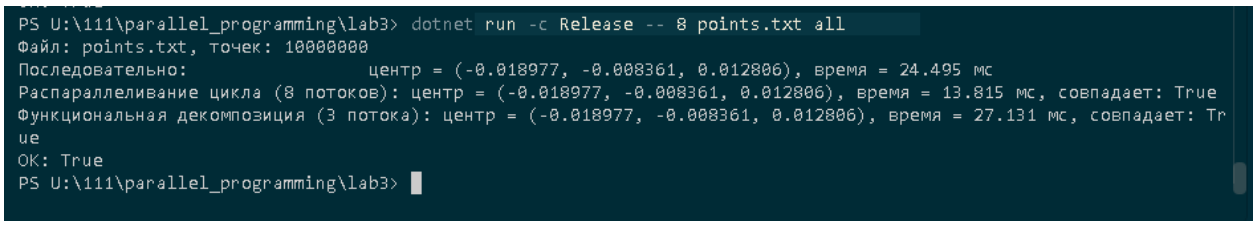


Рис. 4 – запуск обеих версий (режим all) на файле из 10 000 000 точек

Таблица 1 — Сравнение времени выполнения версий программы

Версия	**Потоков**	**Время, мс**	**Совпадает с эталоном**
Последовательная	1	24.495	Да
loop	8	13.815	Да
functional	3	27.131	Да

Выводы

1. Реализованы две параллельные версии вычисления геометрического центра точек из файла: с распараллеливанием цикла (декомпозиция по данным) и с функциональной декомпозицией. Результаты обеих версий совпали с последовательным эталоном (OK: true).

2. Версия loop масштабируется с числом потоков, так как любое количество потоков может обрабатывать свои блоки точек; для объединения результатов достаточно одной редукции частичных сумм после завершения потоков.

3. Версия functional проще для понимания и не требует разбиения данных, но использует ровно три потока (по числу независимых функций), поэтому её ускорение ограничено тремя. Функциональная декомпозиция

особенно полезна, когда части задачи различны по смыслу, а не только по данным.

4. В обеих версиях синхронизация при вычислениях не потребовалась, потому что потоки не пишут в общие данные: частичные результаты хранятся в локальных переменных и в непересекающихся ячейках.